Laboratorio 2 - Variabili e IO

1 Scan

Il programma riportato di seguito legge da **standard input** 4 numeri interi.

- Cosa succede se inserite l'input su righe diverse?
- Cosa succede se inserite meno numeri di quelli richiesti?
- Cosa succede se inserite più numeri di quelli richiesti?
- Cosa succede se inserite un valore diverso da un numero intero (numero reale, lettera, parola, …)?

Scan (2)

```go
package main

import "fmt"

func main() {

    var n1, n2, n3, n4 int

    fmt.Scan(&n1, &n2, &n3, &n4)

    fmt.Println(n1, n2, n3, n4)
}
```

2 Print o Println

Di seguito sono riportati due programmi: in entrambi sono dichiarate 2 variabili `a` e `b` di cui vengono stampati i valori. Notate differenze nell'output prodotto? Cosa cambia tra `Print` e `Println`?

Print

```go
package main

import "fmt"

func main() {
    var a int = 5
    var b float64 = 3.14

    fmt.Print("Valore di a:", a, "capito? Te lo dico due volte:", a, a, "...\n")
    fmt.Print("Valore di b:", b, b, "\n")
}
```

Println

```go
package main

import "fmt"

func main() {
    var a int = 5
    var b float64 = 3.14

    fmt.Println("Valore di a:", a, "capito? Te lo dico due volte:", a, a, "...")
    fmt.Println("Valore di b:", b, b)
}
```

3 Qual è l'output?

Supponendo che l'input fornito da **standard input** sia

```
5 6
```

cosa dovrebbe produrre in output il seguente programma?

```go
package main

import "fmt"

func main() {

    var a, b int
    fmt.Scan(&a, &b)

    var r int
    r = a - b

    fmt.Println(r)

}
```

4 Qual è l'output?

Cosa dovrebbe stampare il seguente programma?

```go
package main

import "fmt"

func main() {
    var a int = 10
```

```
    var b int = 20
    a = a + b
    var c int = a + b
    fmt.Println(c)
}
```

5 Trova l'errore

Questo programma dovrebbe stampare la somma di tre numeri interi `a`, `b` e `c`, ma contiene degli errori. Corregere gli errori e verificare che l'esecuzione produca l'output desiderato.

```
package main

import "fmt"

func main() {
    var a
    a = 10

    var b, d int
    b = 20
    c = 30
    var c int

    var d int = a + b + c
    fmt.Println((d)
}
```

Modulo di un numero e divisione intera

```
package main

import "fmt"

func main() {
    div := 4 / 2
    res := 4 % 2
    fmt.Println(div, res)

    div = 5 / 2
    res = 5 % 2
    fmt.Println(div, res)

    div = 8 / 3
    res = 8 % 3
```

```
    fmt.Println(div, res)

    div = 375 / 10
    res = 375 % 10
    fmt.Println(div, res)
}
```

Utilizzo di altri package (math)

```
package main

import (
    "fmt"
    "math"
)

func main() {
    x := 16.0
    sqrt := math.Sqrt(x)
    sin := math.Sin(math.Pi / 2) // sin(π/2) = 1

    fmt.Println("La radice quadrata di", x, "è", sqrt)
    fmt.Println("Il Seno di π/2 è", sin)
}
```

6 Conversione di tipo

- Qual è il valore di ciascuna delle variabili `r1`, `r2` e `r3`? Di che tipo sono?
- Che differenza c'è tra convertire gli operandi **prima** della divisione e convertire il risultato **dopo** la divisione?
- Cosa succede alla parte decimale di `x` quando viene convertito in `int`?
- Cosa succede se sostituite `float64(a) / float64(b)` con `float64(a) / b`?

```
package main

import "fmt"

func main() {
    a := 5
    b := 2

    r1 := a / b
    r2 := float64(a) / float64(b)
    r3 := float64(a / b)

    fmt.Println("r1:", r1)
```

```
    fmt.Println("r2:", r2)
    fmt.Println("r3:", r3)

    x := 3.9
    y := int(x)

    fmt.Println("x:", x)
    fmt.Println("y:", y)
}
```

6 Area e perimetro rettangolo

Scrivere un programma che legga da **standard input** le misure dell'altezza e della base di un rettangolo e ne calcoli il perimetro e l'area.

Esempio d'esecuzione:

```
$ go run rettangolo.go
Inserisci la base: 20
Inserisci l'altezza: 10
Perimetro = 60
Area = 200
```

7 Area cerchio

Scrivere un programma che legga da **standard input** il raggio di un cerchio e ne calcoli circonferenza e area.

Suggerimento: l'area del cerchio si calcola facendo `raggio x raggio x pi_greco`, mentre la circonferenza facendo `2 x raggio x pi_greco`. Il valore numerico di `pi_greco` è memorizzato nella costante `Pi` del package `math`, a cui ci si può riferire con `math.Pi`.

Esempio d'esecuzione:

```
$ go run cerchio.go
Raggio = 2.5
Circonferenza = 15.707963267948966
Area = 19.634954084936208
```

8 Convertitore Km - miglia

Scrivere un programma che legga da **standard input** una distanza in Km ed effettui la conversione di tale distanza in miglia (1 Km = 0.62137 mi).

Esempio d'esecuzione:

```
$ go run convertitore.go
Distanza (Km) = 12
Distanza (mi) = 7.45644
```

9 Età

Scrivere un programma che legga da **standard input** le età di due persone (espresse in anni) e calcoli:

- la somma delle età inserite;
- la media delle età inserite;
- la media delle età inserite arrotondata per difetto all'intero inferiore;
- la media delle età inserite arrotondata per eccesso all'intero superiore;
- la somma e la media delle età che le due persone avranno fra 10 anni.

Suggerimento: la media arrotondata per difetto può essere calcolata usando la funzione `math.Floor` del package `math` nel seguente modo:

```
var mediaArrotondataDifetto float64 = math.Floor(media)
```

Similarmente, la media arrotondata per eccesso può essere calcolata usando la funzione `math.Ceil` nel seguente modo:

```
var mediaArrotondataEccesso float64 = math.Ceil(media)
```

Esempio d'esecuzione:

```
$ go run calcoloeta.go
Età persona 1 = 15
Età persona 2 = 20
Somma età = 35
Media età = 17.5
Media età arrotondata per difetto all'intero inferiore = 17
Media età arrotondata per eccesso all'intero superiore = 18
Somma età a 10 anni = 55
Media età a 10 anni = 27.5
```

10 Area poligono regolare

Scrivere un programma che legga da **standard input** due numeri interi che chiameremo `n` e `l` e calcoli l'area di un poligono regolare con `n` lati di lunghezza `l`.

Suggerimento: l'area di un poligono regolare può essere calcolata utilizzando le funzioni `math.Pow` (per il calcolo della potenza) e `math.Tan` (per il calcolo della tangente di un angolo) del package `math` applicando la seguente formula:

$$\text{area} = \frac{n \cdot l^2}{4 \cdot \tan\left(\frac{\pi}{n}\right)}$$

Suggerimento: Si consulti la documentazione delle funzioni per verificarne il funzionamento (es. `go doc math.Pow`)

Esempio d'esecuzione:

```
$ go run areapoligono.go
Inserisci il numero di lati del poligono: 6
Inserisci la lunghezza dei lati del poligono: 3
Area calcolata: 23.382685902179844

$ go run areapoligono.go
Inserisci il numero di lati del poligono: 4
Inserisci la lunghezza dei lati del poligono: 3
Area calcolata: 9
```

11 Divisione e resto

Scrivere un programma che prenda in ingresso due numeri e stampi il quoziente ed il resto della divisione intera del primo numero per il secondo.

Esempio di esecuzione:

```
$ go run convertisec.go
Inserisci due numeri: 5 3
Quoziente: 1
Resto: 2
```

12 Da secondi a ore, minuti, secondi

Scrivere un programma che prenda in ingresso un valore intero rappresentante una quantità di tempo in secondi e restituisca il numero di ore, minuti e secondi corrspondenti

Esempio di esecuzione:

```
$ go run convertisec.go
Secondi? 15634
h:m:sec - 4:20:34

$ go run convertisec.go
Secondi? 134
h:m:sec - 0:2:14
```

13 Numero inverito

Scrivi un programma che inverta le cifre di un numero intero a tre cifre inserito da standard input. Esempio: dato `n=123` il programma dovrebbe restituire `321`.

Suggerimento: Puoi ottenere le singole cifre di un numero con operazioni di modulo e divisione intera.

```
$ go run invertinum.go
Numero? 123
Risultato: 321

$ go run invertinum.go
Numero? 453
Risultato: 354
```

14 Distributore automatico

Scrivere un programma che simuli il pagamento di un prodotto acquistato presso un distributore automatico.

Il programma deve leggere da **standard input** il prezzo del prodotto (espresso in centesimi) e la quantità di denaro inserita dall'utente (anch'essa espressa in centesimi).

Supponendo che il denaro inserito sia sufficiente per l'acquisto, il programma deve calcolare il resto e determinare il numero minimo di monete necessarie per restituirlo, utilizzando esclusivamente monete da 2 euro, 1 euro, 50 centesimi, 20 centesimi, 10 centesimi, 5 centesimi, 2 centesimi e 1 centesimo.

Suggerimento: Utilizza le operazioni di divisione intera (`/`) e modulo (`%`) per determinare quante monete di ciascun taglio sono necessarie. Ricorda che 1 euro corrisponde a 100 centesimi.

Esempio d'esecuzione:

```
$ go run distributore.go
Prezzo del prodotto (centesimi): 135
Denaro inserito (centesimi): 500
Resto: 365 centesimi
Monete da 2 euro: 1
Monete da 1 euro: 1
Monete da 50 centesimi: 1
Monete da 20 centesimi: 0
Monete da 10 centesimi: 1
Monete da 5 centesimi: 1
Monete da 2 centesimi: 0
Monete da 1 centesimo: 0
Totale monete: 5
```

15 Decodifica di un messaggio segreto

Scrivere un programma che legga da **standard input** un numero intero di esattamente 6 cifre che rappresenta un messaggio codificato.

Il messaggio originale è composto da tre numeri di due cifre ciascuno, che sono stati codificati utilizzando le seguenti operazioni:

1. Ogni numero originale è stato aumentato di 10.
2. I tre numeri risultanti sono stati concatenati in ordine inverso per ottenere un unico numero di 6 cifre.

Ad esempio, supponiamo che il messaggio originale sia composto dai numeri `44`, `76` e `66`. La codifica avviene nel seguente modo:

1. **Somma di 10:** i tre numeri diventano `54`, `86` e `76`.
2. **Inversione dell'ordine:** i tre numeri vengono riordinati come `76`, `86` e `54`.
3. **Concatenazione:** i tre numeri vengono uniti ottenendo il messaggio codificato `768654`.

Per decodificare il messaggio `768654` è quindi necessario eseguire le operazioni inverse: separare il numero in tre blocchi di due cifre (`76`, `86`, `54`), invertire il loro ordine (`54`, `86`, `76`) e sottrarre 10 a ciascuno, ottenendo il messaggio originale `44`, `76`, `66`.

Scrivere un programma che decodifichi il messaggio, ricostruendo i tre numeri originali e stampandoli nell'ordine corretto.

Si assuma che i tre numeri originali siano compresi tra 10 e 89.

Suggerimento: Utilizza divisione intera e modulo per estrarre i tre blocchi di due cifre. Ricorda che, per ottenere il messaggio originale, devi anche invertire l'ordine dei blocchi e sottrarre 10 a ciascuno.

Esempio d'esecuzione:

```
$ go run decodifica.go
Messaggio codificato: 768654
Primo numero originale: 44
Secondo numero originale: 76
Terzo numero originale: 66

$ go run decodifica.go
Messaggio codificato: 302520
Primo numero originale: 10
Secondo numero originale: 15
Terzo numero originale: 20
```

Esempio d'esecuzione:

```
$ go run decodifica.go
Messaggio codificato: 768654
Primo numero originale: 44
Secondo numero originale: 76
Terzo numero originale: 66

$ go run decodifica.go
Messaggio codificato: 302520
Primo numero originale: 10
Secondo numero originale: 15
Terzo numero originale: 20
```